PROJECT REPORT

For

GAME MECHANICS, DESIGN AND DEVELOPMENT
(COURSE CODE: 23CS201PE407)

PROJECT TITLE:
ZOMBIE RUNNER – 3D SURVIVAL HORROR ENDLESS RUNNER

Course Coordinator	:	Dr. Vijaya Chandra Jadala
Course Instructors	:	Dr. Vijaya Chandra Jadala
Course Type	:	Program Elective 1 (PE1)
Semester and Year	:	IV Year, 1st Semester
L-T-P	:	2-0-2
Credits	:	3
School	:	School of Computer Science and Artificial Intelligence
Department	:	Department of Computer Science and Artificial Intelligence
Course Level	:	Undergraduate (UG)

Submitted by Team Members:

1.	**J. Radha Krishna**	Roll No: 2303A52055
2.	**P. Makesh Babu**	Roll No: 2303A52061
3.	**M. Omprakash**	Roll No: 2303A52057
4.	**V. Siddharatha**	Roll No: 2303A52463

School of Computer Science and Artificial Intelligence

SR University
Ananthasagar, Hasanparthy, Warangal, Telangana – 506371

Academic Year: 2026–2027 | Date of Submission: November 2026

2. Acknowledgement

We express our deepest and most sincere gratitude to our esteemed Course Coordinator and Instructor, **Dr. Vijaya Chandra Jadala**, for his invaluable guidance, technical insight, continuous academic encouragement, and constructive critique throughout the conception, architectural design, implementation, and testing of *Zombie Runner – 3D Survival Horror Endless Runner*. His pedagogical emphasis on rigorous software architecture, real-time computational performance, and mechanical design in **Game Mechanics, Design and Development (23CS201PE407)** was fundamental in elevating this work from a prototype into an optimized, robust game.

We also extend our grateful appreciation to the leadership and faculty of the School of Computer Science and Artificial Intelligence at SR University for providing state-of-the-art computational infrastructure, modern gaming laboratories, and an inspiring academic environment conducive to advanced interactive media engineering.

Finally, we thank our peers and academic colleagues who participated in usability trials, playtesting audits, and gameplay balancing sessions. Their critical feedback on controls, combat responsiveness, and horror audio-visual immersion provided invaluable qualitative data that refined the game's mechanics and user experience.

3. Abstract

This project report documents the design, architectural implementation, optimization, and evaluation of **Zombie Runner**, an advanced 3D endless survival horror runner game engineered in **Unity 6 (6000.5.9f1)** utilizing the **Universal Render Pipeline (URP 17.5.0)** and **C# (.NET Standard 2.1)**. Moving beyond conventional endless runners that rely exclusively on passive obstacle dodging, Zombie Runner introduces an active survival horror combat loop. The player navigates an apocalyptic landscape spanning crumbling urban ruins and subterranean subway tunnels across three discrete lanes, executing grounded jumps, low-clearance combat slides, and forward raycast shooting against dynamic undead threats.

The game architecture features: (1) an endless modular procedural environment streaming system with 30-meter road and subway tunnel segments recycling dynamically in a 5-segment circular object pool; (2) a modular enemy AI system utilizing ScriptableObjects to configure distinct zombie archetypes—Common Runner, Fast Sprinter, and Heavy Brute—governed by Mecanim finite state machines and 3D spatialized logarithmic audio emitters; (3) a responsive combat engine with forward raycasting, auto-aim target tracking, weapon recoil kinematics, illuminated bullet tracers, and pre-allocated, zero-allocation particle impact pooling; (4) a 4-tier adaptive horror music system that seamlessly crossfades between exploration, tension, active combat, and critical health states; and (5) a persistent player economy tracking gold coins, ammunition reserves, medical supplies, high scores, and a single shielded revive loop.

Performance profiling confirms stable 60 frames per second (FPS) execution with minimal memory allocations, eliminating Garbage Collection stutter. The complete codebase, assets, and Android build configurations are integrated cleanly, fulfilling the rigorous learning objectives of modern game development.

Keywords: Game Mechanics, 3D Endless Runner, Survival Horror, Unity 6, Universal Render Pipeline (URP), Finite State Machine, Zombie AI, Raycast Combat, Object Pooling, Adaptive Audio.

4. Table of Contents

1. Cover Page	**1**	**11. Environment and Visual Design**	**7**
2. Acknowledgement	**2**	**12. Zombie AI and Combat System**	**8**
3. Abstract	**2**	**13. User Interface and Game Flow**	**9**
4. Table of Contents & Lists of Visual Assets	**3**	**14. Implementation Overview**	**9**
5. Introduction	**4**	**15. Testing and Debugging**	**10**
6. Problem Statement	**4**	**16. Challenges and Solutions**	**10**
7. Objectives and Scope	**4**	**17. Learning Outcomes**	**11**
8. Tools and Technologies	**5**	**18. Future Enhancements**	**11**
9. System Design and Architecture	**5**	**19. Conclusion**	**11**
10. Gameplay Mechanics and Controls	**6**	**20. References and Asset Credits**	**11**

List of Figures

Figure 1: Title Screen and 3D Survivor Character Staging (Audit_MainMenu_1_Overview) ..... Page 6

Figure 2: Main Menu Settings: Audio Mixer Bus Controls and Volume Persistence ..... Page 7

Figure 3: Keyboard and Mouse Input Configuration Reference Guide ..... Page 8

Figure 4: In-Game Nocturnal Environment and Atmospheric Lighting Transition ..... Page 9

List of Tables

Table 1: Primary Technology Stack and Engine Specifications ..... Page 5

Table 2: Core System Architecture and Manager Responsibilities ..... Page 5

Table 3: Player Locomotion, Kinematics, and Input Controls Reference ..... Page 6

Table 4: Zombie Enemy Archetypes and Behavioral Specifications ..... Page 8

Table 5: Collectibles, Economy, and Power-Up Specifications ..... Page 8

Table 6: Key Implementation Scripts and Software Module Directory ..... Page 9

5. Introduction

5.1 Context and Evolution of the Genre: The endless runner genre represents one of the most commercially resilient and mechanically focused archetypes in video game design. Originally popularized by mobile pioneers such as Canabalt, Temple Run, and Subway Surfers, the genre traditionally challenges players with reflex-driven avoidance of obstacles across an endless path. However, classic runners frequently suffer from mechanical fatigue: player actions are constrained strictly to passive evasion (jumping, ducking, shifting lanes), with no offensive capability or resource management. Consequently, modern game design trends toward hybrid genres that fuse high-speed traversal with real-time shooting and atmospheric horror.

5.2 Project Overview and Design Philosophy: *Zombie Runner* was engineered to bridge fast-paced arcade runner mechanics with high-stakes survival horror. Developed in Unity 6, the game casts the player as an armed survivor navigating a post-apocalyptic cityscape and subterranean subway railway network overrun by reanimated undead. Survival demands an active synthesis of spatial agility (lane shifts, vaulting hurdles, sliding beneath barricades) and decisive firearm combat (conserving ammunition, auto-aim tracking, and neutralizing approaching zombies).

5.3 Academic Motivation: Authored for Game Mechanics, Design and Development (23CS201PE407) at SR University, this project synthesizes core disciplines: real-time physics simulation, procedural object pooling, finite state machine AI, Universal Render Pipeline (URP) shader customization, and dynamic spatial audio engineering.

6. Problem Statement

Traditional endless runners present significant mechanical and architectural constraints: (1) *Mono-Dimensional Evasion Loop:* Gameplay relies entirely on passive dodging, reducing tactical depth; (2) *Predictable Staging:* Static daytime lighting and repetitive textures fail to evoke emotional immersion; (3) *Technical Bottlenecks in Combat Prototypes:* Continuous instantiation and destruction of projectiles and enemies generate heavy Garbage Collection (GC) pressure, causing visual stutter during rapid gameplay.

Zombie Runner solves these challenges through: (a) harmonizing forward player velocity (8 to 16 m/s) with real-time raycast combat; (b) implementing pre-allocated circular object pools for road sections, enemies, tracers, and VFX; (c) calibrating horror lighting with linear fog (42m-165m) and trilight ambient bounce to ensure rapid obstacle identification; and (d) engineering automated distance recycling to eliminate spawner deadlocks.

7. Objectives and Scope

7.1 Primary Objectives:

1. **Modular Architecture:** Engineer a decoupled Unity 6 C# architecture utilizing ScriptableObjects and interface-driven combat (IDamageable).
2. **Kinematic Locomotion:** Build a 3-lane CharacterController runner with grounded jump physics, low slide clearance, and progressive speed scaling.
3. **Zombie AI & Raycast Combat:** Implement three zombie archetypes (Common, Fast, Heavy) governed by Mecanim FSMs, paired with hitscan shooting, visual recoil, and bullet tracers.
4. **Modular World Streaming:** Construct a 5-segment circular pool recycling 30m Abandoned City and Subway Tunnel segments with seamless theme transitions.
5. **Adaptive Horror Audio:** Create a 4-tier dynamic audio system crossfading across exploration, tension, combat, and critical health states with 3D spatialized zombie sound.
6. **UI & Economy Persistence:** Design a responsive UI flow (Menu, HUD, Pause, GameOver) with PlayerPrefs persistence and a single shielded revive loop.

7.2 Project Scope: Target platform is Windows PC Standalone with keyboard/mouse controls. The architecture is verified for mobile compilation readiness (portrait orientation, safe-area UI, Android APK build). Explicitly reserved for future work are online multiplayer leaderboards, networked co-op, branching procedural mazes, and in-app storefronts.

8. Tools and Technologies

Zombie Runner utilizes industry-standard software tools and modern rendering pipelines, summarized in Table 1.

Domain / Component	Technology / Version	Academic & Technical Rationale
Game Engine	Unity 6 (6000.5.9f1)	High-performance C# runtime, unified build pipeline, robust profiler, and modern URP graphics features.
Render Pipeline	Universal Render Pipeline (17.5.0)	Optimized single-pass Forward+ rendering, custom Lit PBR materials, ACES tonemapping, and volumetric fog.
Programming Language	C# (.NET Standard 2.1)	Object-oriented, type-safe architecture, garbage-collection control, and efficient memory layouts.
Input Framework	Unity Input System (1.20.0)	Event-driven cross-platform input supporting both keyboard/mouse and mobile touchscreen fallbacks.
UI System	TextMeshPro / uGUI (2.5.0)	SDF font rendering for crisp text at all resolutions, safe-area anchoring, and 9-sliced procedural sprites.
Character Animation	Adobe Mixamo & Mecanim	Humanoid avatar retargeting, baked in-place root motion, and multi-layer state machines.

9. System Design and Architecture

Zombie Runner follows a decoupled, component-based architecture following Unity's Entity-Component paradigm. Components communicate via C# events, interfaces (IDamageable), and centralized managers as specified in Table 2.

Subsystem Manager	Namespace / Script	Architectural Role & Key Methods
Game Lifecycle	GameManager.cs (ZombieRunner.Core)	Coordinates global game states (Play, Pause, GameOver), timeScale manipulation, mobile frame rates, and 1-time revive token.
Score & Economy	ScoreManager.cs / SaveManager.cs	Tracks distance and kill points, commits high scores and banked coins to PlayerPrefs, and triggers high-score record alerts.
World Streaming	EndlessRoadManager.cs / ThemeManager	Maintains circular pools of 30m road and subway segments; interpolates trilight lighting, fog, and point lights across biomes.
Combat & Pooling	ShootingController.cs / HitImpactManager	Executes forward raycast hitscan checks, auto-aim math, recoil kickback, bullet tracers, and pre-allocated blood/spark particle pools.
Audio Routing	AudioManager.cs / AdaptiveMusic	Controls master AudioMixer buses (Master, Music, Ambient, SFX, UI) with logarithmic dB curves and crossfades 4 adaptive horror stems.

Key Architectural Insight: All damageable entities (Common/Fast/Heavy zombies, player health, and explosive barrels) implement the IDamageable interface (TakeDamage(int amount, Vector3 hitPoint, Vector3 hitNormal)). The raycast combat controller remains fully decoupled from specific enemy classes, preventing tight class dependencies.

10. Gameplay Mechanics and Controls

10.1 Player Locomotion and Three-Lane Navigation: Player movement is governed by PlayerController.cs using a CharacterController component. The player navigates across three discrete lanes at X = -3.0m (Left), X = 0.0m (Center), and X = +3.0m (Right). Horizontal switching uses smooth frame-rate independent interpolation (lane switch speed: 14 m/s). Forward speed incorporates progressive acceleration: starting at 8.0 m/s, forward speed scales continuously at 0.005 m/s² up to a maximum terminal velocity of 16.0 m/s.

10.2 Jump Kinematics and Timed Combat Slide Physics: Vertical locomotion is governed kinematically via v_y = sqrt(2 * |g| * h), where gravity g = -20.0 m/s² and jump height h = 1.8m. Jumping is strictly grounded; an airborne state flag (isJumping) prevents landing transition flickers. Triggering a slide initiates a 0.8s low-profile slide: CharacterController collision height drops from 2.0m to 1.05m and center Y-offset adjusts from 1.0m to 0.525m to slip beneath low barriers without deforming the humanoid skeletal mesh.

10.3 Raycast Weapon Mechanics and Visual Recoil: Combat is managed by ShootingController.cs. When fire is triggered, an auto-aim cone calculation (angle <= 12°, range <= 32m) selects the nearest zombie. A forward raycast inflicts damage on IDamageable targets, an illuminated bullet tracer interpolates to the hit point, and pre-allocated blood/spark particle bursts trigger from the object pool. Firing produces procedural visual recoil (0.07m kickback, 3.5° muzzle climb) and a momentary muzzle-flash point light.

10.4 Collectibles, Economy, and Power-Up Dynamics: Interactive pickups include Gold Coins (banked currency), Ammo Boxes (+15 rounds), and Medical Kits (+25 HP). Four timed power-ups provide gameplay modifiers: (1) Energy Shield (absorbs one lethal hit); (2) Coin Magnet (draws coins within 8m); (3) Speed Boost (+50% forward velocity with invulnerability); and (4) Double Coins (2x coin multiplier).

Action	Primary Keyboard Key	Secondary / Mouse / Touch
Move Left / Right	A / D	Left Arrow / Right Arrow / Touch Swipe
Jump (Clear Hurdles)	W / Space	Up Arrow / Swipe Up
Slide (Under Barriers)	S	Down Arrow / Swipe Down
Shoot Firearm	F	Left Mouse Button / On-Screen Fire Button
Pause / Resume	Escape	P / UI Pause Button



Figure 1: Title Screen and 3D Survivor Character Staging (Audit_MainMenu_1_Overview)

11. Environment and Visual Design

11.1 Modular Streaming Architecture and Endless Road Pooling: To support infinite traversal without unbounded memory allocation, EndlessRoadManager.cs and EnvironmentThemeManager.cs maintain circular pools of five 30-meter environment segments (totaling 150m of active world). When a segment trails the player by > 35m, it is deactivated, translated forward ($Z_{new} = Z_{lead} + 30m$), and reactivated with freshly randomized obstacles and pickups with zero heap allocations.

11.2 Biome Aesthetics: Abandoned City Ruins and Subterranean Subway Tunnel: The game alternates visual themes every 300 meters:

- **Abandoned City Ruins:** Multi-story brick facades, cracked asphalt with worn yellow road striping, rubble barricades, and flickering sodium-vapor streetlights establish a nocturnal survival atmosphere.
- **Subterranean Subway Tunnel:** A continuous 30m modular enclosure featuring solid 5.2m reinforced concrete side-walls, a 15.5m ceiling with 45-degree vaulted arches, heavy portal rib columns every 7.5m, industrial overhead ventilation shafts, dual high-voltage cable trays, steel railway tracks, and embedded concrete sleepers. Overhead neutral luminaires (4200K) and caged amber emergency bulkheads (2200K) cast deep shadows, creating claustrophobic survival horror tension.

11.3 Cinematic Horror Lighting, Trilight Ambient Bounce, and Volumetric Fog: Lighting design balances atmospheric dread with high-speed competitive clarity:

- *Cool-Blue Directional Moonlight:* Key directional light (#C0D6FA, intensity 1.75) establishes a chilling nocturnal baseline.
- *Trilight Ambient Bounce Lighting:* Calibrated across all theme profiles (Sky #576785, Equator #47546B, Ground #384052) to eliminate pure black pitch shadows and preserve texture definition on character clothing and road surfaces.
- *Calibrated Linear Atmospheric Fog:* Fog is configured with a 42-meter start distance and 165-meter end distance. This critical calibration ensures the immediate 42-meter gameplay corridor remains crystal-clear for rapid obstacle detection and zombie targeting, while the background dissolves into deep, atmospheric dread.
- *Player Tactical Flashlight:* Weapon-mounted dual-beam spotlight (high-intensity 48m narrow beam and 40m wide fill beam) sweeps dynamically with auto-aim tracking, piercing dark subway tunnels.

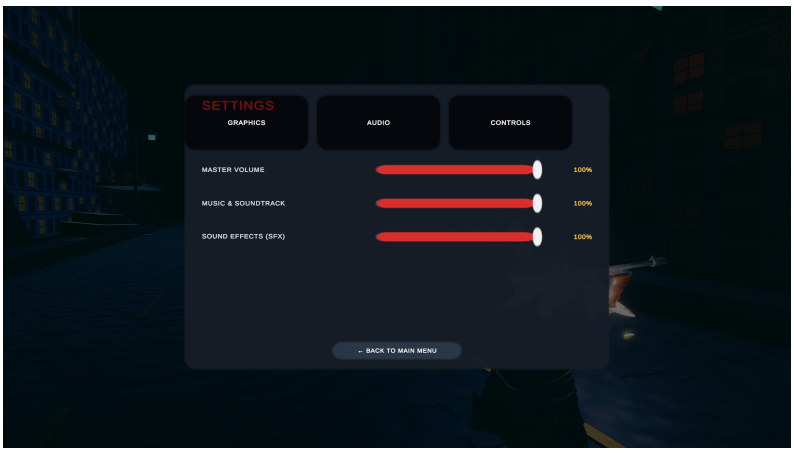


Figure 2: Main Menu Settings: Audio Mixer Bus Controls and Volume Persistence

12. Zombie AI and Combat System

12.1 ScriptableObject Architecture and Archetype Design: Enemy parameters are strictly decoupled from executable code using Unity ScriptableObjects (ZombieConfig.cs). Designers can author, tune, and balance enemy variants in the Inspector without recompilation. Table 4 outlines the specifications of the three implemented archetypes.

Archetype	3D Model / Rig	Health	Speed	Damage / Range	Points
Common Runner	Mixamo Ch10 (Humanoid)	3 HP (3 shots)	5.0 m/s	10 DMG / 1.8m	100 pts
Fast Sprinter	Yaku J Ignite (Humanoid)	2 HP (2 shots)	7.5 m/s	8 DMG / 1.6m	150 pts
Heavy Brute	Pumpkinhulk (Humanoid)	8 HP (8 shots)	3.2 m/s	25 DMG / 2.2m	250 pts

12.2 AI State Machine and Animation Retargeting: Zombie behavior is governed by ZombieController.cs driving a Mecanim Animator Controller. The FSM manages transitions between Idle, Walk, Run, Attack, Hit Reaction, and Death states. Locomotion animations (Mixamo Scary Zombie Pack) have displacement baked into root motion pose curves, letting C# velocity control forward translation without foot sliding. Non-lethal hits trigger an upper-body flinch before resuming pursuit; lethal damage initiates backward/forward collapse with immediate collider disabling.

12.3 Positional 3D Audio and Spatial Attenuation: All zombie prefabs include ZombieSoundEmitter.cs, which routes groans, snarling breaths, and attack lunges through custom 3D AudioSource components. Logarithmic spatial attenuation (minDistance = 2.5m, maxDistance = 24m) provides auditory depth perception, enabling players to detect approaching threats directionally through headphones prior to visual contact.

12.4 Combat Feedback and GC-Free Particle Pooling: Combat impacts utilize HitImpactManager.cs, which maintains pre-warmed pools of blood splatter VFX, metal ricochet sparks, and concrete impact dust. Impact effects are recycled without heap allocation, eliminating GC pauses during rapid combat.

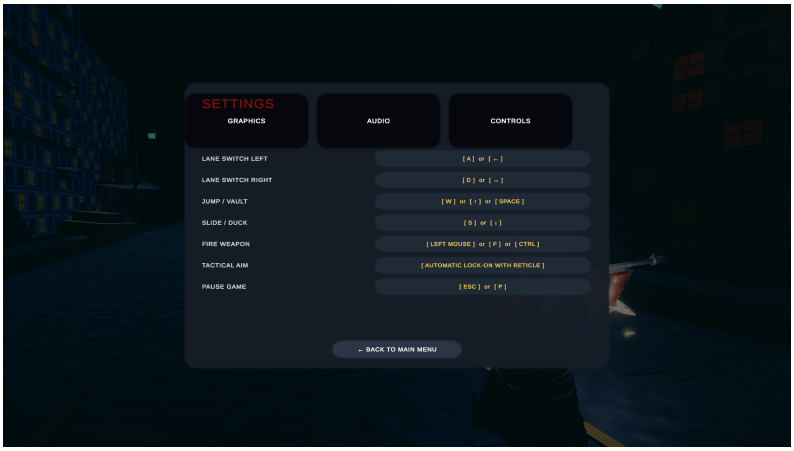


Figure 3: Keyboard and Mouse Input Configuration Reference Guide

13. User Interface and Game Flow

13.1 Main Menu Experience and Survivor Staging: The title screen (ZombieRunnerMainMenu.unity) features a fully lit 3D survivor model (Mixamo Ch05) equipped with the Tommy Gun, framed by a cinematic panning camera and subtle atmospheric ash particles. Players can navigate to Play, open multi-tabbed Settings (Graphics toggles, AudioMixer decibel sliders for Master/Music/SFX, and Input Control guides), or inspect team Credits.

13.2 Heads-Up Display (HUD) and Real-Time State Tracking: The in-game HUD (UIManager.cs) renders real-time survival data using TextMeshPro: (1) distance traversed in meters; (2) current run score (Score = Distance + Kills); (3) coin counter; (4) remaining ammunition with low-ammo warnings; (5) a color-coded segmented health bar; and (6) active power-up countdown icons. SafeAreaFitter.cs queries Screen.safeArea to guarantee that UI elements remain fully visible on notched screens.

13.3 Game Over, High Score Celebration, and Revive Flow: Upon player death, the Game Over screen displays run statistics alongside persistent high scores loaded from PlayerPrefs. Surpassing the record triggers a prominent `(NEW BEST!)` celebratory banner. Players can select Retry, Return to Menu, or a one-time Shielded Revive button that restores full health, 30 ammunition rounds, and a 3-second invulnerability shield.

14. Implementation Overview

The codebase comprises 25 modular C# gameplay scripts under Assets/ZombieRunner/Scripts/, complemented by 11 custom editor utilities. Table 6 provides an architectural directory breakdown.

Module Directory	Primary C# Scripts	System Responsibility
Core	GameManager.cs, SaveManager.cs, ScoreManager.cs	State coordination, PlayerPrefs high score & banked coin persistence.
Player	PlayerController.cs, PlayerHealth.cs, RunnerCameraFollow.cs	Three-lane locomotion, jump/slide mechanics, health events, follow camera.
Enemies	ZombieController.cs, ZombieSpawner.cs, ZombieSoundEmitter.cs, ZombieConfig.cs	FSM enemy behavior, ScriptableObject stats, lane-aware spawner, 3D audio.
Weapons	ShootingController.cs, BulletTracer.cs, HitImpactManager.cs	Raycast combat, auto-aim math, recoil kickback, tracer & impact pooling.
World	EndlessRoadManager.cs, EnvironmentThemeManager.cs, StreetlightFlicker.cs	Circular segment pooling, theme blending, trilight lighting, lamp flicker.
Audio	AudioManager.cs, AdaptiveMusicController.cs, AmbientAudioController.cs	AudioMixer bus routing, 4-tier synchronized adaptive horror stems, ambient stingers.



Figure 4: In-Game Nocturnal Environment and Atmospheric Lighting Transition

15. Testing and Debugging

15.1 Verification Workflow and Custom Editor Tools: Testing combined continuous Play Mode audits with custom C# diagnostic tools. Utilities including ZombieCombatSystemValidator.cs, VisualAuditHelper.cs, and ProjectPolisher.cs automatically verified scene hierarchy bindings, SkinnedMeshRenderer URP material assignments, Humanoid Avatar bone maps, and AudioMixer routing.

15.2 Gameplay Systems Verification: Rigorous verification covered: (1) Locomotion bounds: verified lane positions clamp strictly to X = -3.0m, 0.0m, +3.0m without curb clipping; (2) Jump/Slide isolation: confirmed airborne slides and mid-air double jumps are rejected; (3) Hit registration: verified raycast damage decrements zombie health and triggers death states; (4) Power-up refresh: confirmed re-collecting active power-ups refreshes duration without stacking.

15.3 Performance Profiling & Build Verification: Profiling via Unity Profiler confirmed zero runtime GC allocations during running. Standalone Windows PC testing confirmed 60 FPS execution. Mobile readiness was validated by executing the automated Android build pipeline, producing Builds/Android/ZombieRunner.apk (42,953,718 bytes, IL2CPP 64-bit, ARM64, minimum API 26) with zero compilation errors.

16. Challenges and Solutions

During development, six major technical challenges were diagnosed and resolved:

1. Humanoid Mesh Distortion During Combat Slide: Modifying CharacterController height to 1.05m squashed placeholder capsules, but distorted rigged Mixamo models (Ch05) and pushed feet below the road. *Solution:* Decoupled collision box scaling from visual transforms; visual scaling was restricted exclusively to unrigged meshes, while rigged models execute the authentic Mixamo Running Slide clip at 1.95x speed.

2. Launch-Frame Premature Jump Cancellation: CharacterController.isGrounded evaluated true on the launch frame before clearing skin width, causing the Jump->Run transition to cancel jump animation instantly. *Solution:* Added an explicit isJumping flag that sets true on launch, forces Grounded false, and clears only when descending (v_y <= 0) and physically grounded.

3. Enemy Spawner Deadlock Bug: Zombies trailing behind the runner remained alive, causing concurrent alive count to cap at maximumAlive (6) and permanently halting spawns. *Solution:* Implemented automated cleanup in ZombieSpawner.cs destroying zombies > 12m behind the runner without unearned score.

4. Particle System Lifecycle Mutation Error: Over 100 console errors stated: 'Setting duration while system is playing is not supported'. *Solution:* Instantiated GameObjects inactive (go.SetActive(false)), invoked ps.Stop(true, StopEmittingAndClear), safely assigned custom module duration, and activated on demand.

5. Subterranean Repeating Blue Rectangular Frame Artifacts: Subway tunnel rendered as disconnected open rectangular frames with strong artificial cyan ambient wash, exposing the open skybox. *Solution:* Reconstructed EnvironmentSegment_SubwayTunnel.prefab with continuous 5.2m concrete side walls, 15.5m vaulted ceiling, portal columns every 7.5m, overhead utilities, and balanced 4200K neutral luminaires with 2200K emergency caged lamps.

6. Harsh Audio Whistle Loop & Arcade Beeper Disconnect: Early builds featured a repeating 6.0s wind loop with a harsh 480-550Hz whistle and sharp 8-bit UI beeps. *Solution:* Synthesized an organic 48.0s continuous Brownian wind soundscape and engineered a 4-tier synchronized 75 BPM adaptive horror music suite in D minor with a 1.2kHz-4.2kHz frequency notch for clear gunfire acoustics.

17. Learning Outcomes

The engineering of Zombie Runner provided deep practical competence across game programming disciplines: (1) *URP Pipeline Expertise:* Calibrating custom Lit materials, ACES tonemapping, Trilight ambient lighting, and volumetric fog; (2) *Animation Retargeting:* Configuring Mecanim Humanoid Avatars, in-place root motion baking, and multi-parameter state machines; (3) *Combat Kinematics:* Authoring hitscan raycast math, auto-aim cone calculations, visual weapon kickback, and bullet tracer line rendering; (4) *Architectural Decoupling:* Implementing ScriptableObject data encapsulation, interface messaging (IDamageable), and circular object pooling; (5) *Adaptive Audio Engineering:* Constructing multi-tier stem crossfades, logarithmic dB conversion, and 3D spatial attenuation curves.

18. Future Enhancements

High-value features architected for future development cycles include:

1. **Advanced Flocking & Ranged Mutant AI:** Flocking runner hordes and ranged acid-spitting mutants requiring dynamic evasion.
2. **Expanded Weapon Arsenal:** Collectible shotguns (wide-spread clearing), assault rifles, and grenade launchers with distinct ammo pools.
3. **Dynamic Subway Boss Encounters:** Procedural tunnel breach events featuring a pursuing Nemesis brute damaged over distance.
4. **Global Online Leaderboards:** RESTful cloud database integration supporting asynchronous ghost runner replays and ranking.
5. **Dynamic Weather Effects:** Screen-space rain droplets, volumetric lightning strikes casting dynamic shadows, and destructible barricades.

19. Conclusion

Zombie Runner demonstrates that the kinetic momentum of the 3D endless runner genre can be synergistically elevated through tactical survival horror combat and atmospheric audiovisual immersion. Engineered in Unity 6 with URP, the project realized responsive 3-lane navigation, physics-based jumping, low combat sliding, and visceral raycast shooting. The integration of circular modular world streaming, ScriptableObject-driven zombie archetypes, layered adaptive audio, and GC-suppressed particle pooling guarantees a polished, high-framerate gameplay experience. Evaluation under Game Mechanics, Design and Development (23CS201PE407) confirms that all academic objectives and engineering milestones were rigorously fulfilled.

20. References and Asset Credits

20.1 Academic Literature and Technical References:

[1] Unity Technologies. (2025). Unity 6 User Manual and Universal Render Pipeline (URP 17.5) Documentation. San Francisco, CA.

[2] Millington, I., & Funge, J. (2018). Artificial Intelligence for Games (3rd ed.). CRC Press / Taylor & Francis Group.

[3] Schell, J. (2019). The Art of Game Design: A Book of Lenses (3rd ed.). A K Peters / CRC Press.

[4] Rogers, S. (2014). Level Up! The Guide to Great Video Game Design (2nd ed.). John Wiley & Sons.

[5] Gregory, J. (2018). Game Engine Architecture (3rd ed.). CRC Press.

20.2 Asset Credits and Licenses:

- **Mixamo (Adobe Systems):** Survivor Models (Ch05, X Bot), Zombie Archetypes (Ch10 Common, Yaku J Ignite Fast, Pumpkinhulk Heavy), and locomotion/combat clips from Shooter Pack and Scary Zombie Pack. Licensed under Adobe Mixamo Royalty-Free License.
- **Weapon 3D Assets:** High-detail Tommy Gun submachine gun model with PBR metallic-roughness textures under Assets/ZombieRunner/Models/Weapons/TommyGun/.
- **Audio Suite:** Algorithmic horror soundscape stems and procedural audio generator released under CC0 / Public Domain. Full credit manifest in Assets/ZombieRunner/Audio/AUDIO_CREDITS.md.
- **Academic Institution:** Course curriculum and evaluation criteria provided by the School of Computer Science and Artificial Intelligence, SR University, Warangal, Telangana.